

1. Introduction to JavaScript

What is JavaScript?

JavaScript is a programming language used to make websites interactive and dynamic.

With JavaScript, you can:

- Respond to button clicks
- Show or hide content
- Validate forms
- Create sliders and menus
- Fetch data from APIs
- Build web applications

Real-World Analogy

Think of a website as a human body:

- **HTML** = Skeleton (structure)
- **CSS** = Clothes & appearance (style)
- **JavaScript** = Brain & muscles (behavior and actions)

Without JavaScript, a website can display information, but it cannot interact with users effectively.

Example

HTML:

```
<button>Click Me</button>
```

With JavaScript, the button can perform an action when clicked.

```
console.log("Button clicked!");
```

Where is JavaScript Used?

- Frontend (Web Browser)
- Backend (Node.js)
- Mobile Apps
- Desktop Apps
- Games
- APIs

Why Learn JavaScript?

- Core language of web development
- Required for modern frontend frameworks like React
- Used in backend development with Node.js

- Essential for becoming a Full-Stack Engineer

Practice

Answer these questions:

1. What is JavaScript?
2. What are the roles of HTML, CSS, and JavaScript in a website?
3. Give three examples of what JavaScript can do on a website.

3. Variables (`var`, `let`, `const`)

What is a Variable?

A variable is a named container that stores data in memory so that you can use it later in your program.

Instead of writing the same value repeatedly, you store it in a variable and refer to it by its name.

Example

Without variables:

```
console.log("Mainul");  
console.log("Mainul");  
console.log("Mainul");
```

With variables:

```
let name = "Mainul";  
  
console.log(name);  
console.log(name);  
console.log(name);
```

This makes code easier to read, update, and maintain.

Real-World Analogy

Imagine you have three boxes:

```
Box Name    → Mainul  
Box Age     → 20  
Box Country → Bangladesh
```

Each box has:

- A label (variable name)
- A value (stored data)

In JavaScript:

```
let name = "Mainul";  
let age = 20;  
let country = "Bangladesh";
```

Variable Declaration Syntax

```
let variableName = value;
```

Example:

```
let age = 20;
```

- `let` = keyword
 - `age` = variable name
 - `=` = assignment operator
 - `20` = value
-

let

Use `let` when the value can change later.

Example

```
let age = 20;  
  
age = 21;  
  
console.log(age);
```

Output:

21

The value changed from 20 to 21.

Real-Life Example

```
let score = 0;  
  
score = 10;  
score = 20;
```

A player's score changes during a game, so `let` is appropriate.

const

Use `const` when the value should never change.

Example

```
const country = "Bangladesh";  
console.log(country);
```

Output:

Bangladesh

Error Example

```
const country = "Bangladesh";  
country = "India";
```

Output:

Error

Because a `const` variable cannot be reassigned.

Real-Life Example

```
const birthYear = 2005;
```

Your birth year does not change, so `const` is suitable.

`var`

Before ES6 (2015), developers mainly used `var`.

Example:

```
var name = "Mainul";
```

Today, most developers prefer:

```
let  
const
```

because they are safer and more predictable.

Differences Between `var`, `let`, and `const`

Feature	var	let	const
Can reassign value	✓	✓	✗
Can redeclare variable	✓	✗	✗
Modern usage	✗ Rare	✓ Yes	✓ Yes

Example:

```
var age = 20;
var age = 21;
```

Works.

```
let age = 20;
let age = 21;
```

Error.

Variable Naming Rules

Valid

```
let name = "Mainul";
let age = 20;
let userName = "Rabby";
let my_age = 20;
let $price = 100;
```

Invalid

```
let lname = "Mainul";
let user-name = "Rabby";
let let = 10;
```

Naming Best Practices

Use meaningful names.

✓ Good

```
let studentName = "Mainul";
let totalMarks = 500;
let isLoggedIn = true;
```

✗ Bad

```
let a = "Mainul";
```

```
let x = 500;  
let z = true;
```

Common Beginner Mistakes

Forgetting Quotes



```
let name = Mainul;
```



```
let name = "Mainul";
```

Reassigning a const



```
const age = 20;  
age = 21;
```

Using Meaningless Names



```
let a = 10;  
let b = 20;
```



```
let productPrice = 10;  
let productQuantity = 20;
```

Interview Questions and Answers

1. What is a variable in JavaScript?

Answer:

A variable is a named container used to store data that can be used later in a program.

2. Why do we use variables?

Answer:

Variables help store, manage, and reuse data without writing the same value repeatedly.

3. What are the three ways to declare variables in JavaScript?

Answer:

```
var  
let  
const
```

4. Which variable declaration is most commonly used today?

Answer:

`let` and `const` are most commonly used in modern JavaScript.

5. What is the difference between `let` and `const`?

Answer:

- `let` allows reassignment.
- `const` does not allow reassignment.

Example:

```
let age = 20;  
age = 21; // Allowed  
  
const country = "Bangladesh";  
country = "India"; // Error
```

6. Can a `const` variable be declared without a value?

Answer:

No.



```
const age;
```



```
const age = 20;
```

7. Which should be your default choice: `let` or `const`?

Answer:

Use `const` by default.

Use `let` only when the value needs to change.

8. Is `var` still supported?

Answer:

Yes, JavaScript still supports `var`, but modern developers usually prefer `let` and `const`.

9. Can variable names start with a number?

Answer:

No.

✗

```
let 1name = "Mainul";
```

✓

```
let name1 = "Mainul";
```

10. What will be the output?

```
let age = 20;
```

```
age = 25;
```

```
console.log(age);
```

Answer:

25

Practice

Create:

name

```
age
country
isStudent
```

Use `const` and `let` appropriately and tell me your code.

4. Data Types

What is a Data Type?

A data type tells JavaScript what kind of value is stored in a variable.

Think of a variable as a box and the data type as the type of item inside that box.

```
let name = "Mainul";
let age = 20;
let isStudent = true;
```

Here:

- `"Mainul"` → `String`
- `20` → `Number`
- `true` → `Boolean`

JavaScript needs to know the type of data so it can work with it correctly.

Why Are Data Types Important?

Imagine you have:

```
let age = 20;
let name = "Mainul";
```

You can do math with `age`:

```
console.log(age + 5);
```

Output:

```
25
```

But doing math with a name doesn't make sense:

```
console.log(name + 5);
```

Output:

Because JavaScript treats strings and numbers differently.

Primitive Data Types

The most important data types for beginners are:

1. String
 2. Number
 3. Boolean
 4. Undefined
 5. Null
-

1. String

A string is text enclosed in quotes.

Example

```
let name = "Mainul";  
let country = 'Bangladesh';
```

Valid Quotes

```
let firstName = "Mainul";  
let lastName = 'Rabby';
```

Output

```
console.log(name);  
Mainul
```

Real-Life Examples

```
let city = "Dhaka";  
let email = "mainul@gmail.com";  
let course = "JavaScript";
```

2. Number

A number represents numeric values.

Integer

```
let age = 20;  
let marks = 95;
```

Decimal

```
let price = 99.99;  
let height = 5.8;
```

Examples

```
let a = 10;  
let b = 5;  
  
console.log(a + b);
```

Output:

```
15
```

3. Boolean

A boolean has only two possible values:

```
true  
false
```

Example

```
let isStudent = true;  
let isLoggedIn = false;
```

Real-Life Examples

```
let hasPermission = true;  
let isAdmin = false;  
let isOnline = true;
```

Think of a light switch:

```
ON   = true  
OFF  = false
```

4. Undefined

A variable that has been declared but not assigned a value has the value `undefined`.

Example

```
let age;  
console.log(age);
```

Output:

undefined

JavaScript says:

"The variable exists, but no value has been assigned."

5. Null

`null` means intentionally empty.

Example

```
let selectedUser = null;
```

This means:

"There is currently no selected user."

Difference Between Null and Undefined

Undefined

```
let age;
```

Meaning:

No value assigned yet.

Null

```
let age = null;
```

Meaning:

I intentionally set it to empty.

Checking Data Types

JavaScript provides the `typeof` operator.

String

```
let name = "Mainul";  
console.log(typeof name);
```

Output:

```
string
```

Number

```
let age = 20;  
console.log(typeof age);
```

Output:

```
number
```

Boolean

```
let isStudent = true;  
console.log(typeof isStudent);
```

Output:

```
boolean
```

Undefined

```
let city;  
console.log(typeof city);
```

Output:

```
undefined
```

Null (Important Interview Question)

```
let value = null;  
console.log(typeof value);
```

Output:

object

This is a well-known JavaScript bug that remains for historical reasons.

Summary Table

Data Type	Example
String	"Mainul"
Number	20, 99.99
Boolean	true, false
Undefined	let age;
Null	let age = null;

Common Beginner Mistakes

Forgetting Quotes



```
let name = Mainul;
```



```
let name = "Mainul";
```

Using String Instead of Number

```
let age = "20";
```

This is a string, not a number.

```
console.log(typeof age);
```

Output:

```
string
```

Interview Questions and Answers

1. What is a data type?

Answer:

A data type defines the kind of value stored in a variable.

2. Name five primitive data types in JavaScript.**Answer:**

- String
 - Number
 - Boolean
 - Undefined
 - Null
-

3. What is a string?**Answer:**

A string is a sequence of characters enclosed in quotes.

Example:

```
let name = "Mainul";
```

4. What is a boolean?**Answer:**

A boolean stores either `true` or `false`.

Example:

```
let isStudent = true;
```

5. What is the difference between `null` and `undefined`?**Answer:**

- `undefined` = variable declared but no value assigned.
 - `null` = intentionally assigned an empty value.
-

6. What does `typeof` do?**Answer:**

It returns the data type of a value.

Example:

```
typeof "Mainul";
```

Output:

```
string
```

7. What is the output?

```
let age = 20;  
console.log(typeof age);
```

Answer:

```
number
```

8. What is the output?

```
let name = "Mainul";  
console.log(typeof name);
```

Answer:

```
string
```

9. What is the output?

```
let isStudent = true;  
console.log(typeof isStudent);
```

Answer:

```
boolean
```

10. Why does `typeof null` return "object"?

Answer:

It is a historical bug in JavaScript that has been kept for backward compatibility.

Practice

Predict the output:

```
let name = "Mainul";  
let age = 20;  
let isStudent = true;  
let city;  
  
console.log(typeof name);  
console.log(typeof age);  
console.log(typeof isStudent);  
console.log(typeof city);
```

Write your answers before moving to the next topic.

5. Operators

What is an Operator?

An operator is a symbol that performs an operation on one or more values.

Real-World Analogy

Think of operators as tools.

```
+ → Add  
- → Subtract  
* → Multiply  
/ → Divide
```

Just as a calculator uses symbols to perform calculations, JavaScript uses operators to perform operations on data.

Types of Operators

For beginners, focus on:

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators

1. Arithmetic Operators

Used for mathematical calculations.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus (Remainder)
**	Exponent (Power)

Addition (+)

```
let a = 10;  
let b = 5;  
  
console.log(a + b);
```

Output:

15

Subtraction (-)

```
let a = 10;  
let b = 5;  
  
console.log(a - b);
```

Output:

5

Multiplication (*)

```
let a = 10;  
let b = 5;  
  
console.log(a * b);
```

Output:

50

Division (/)

```
let a = 10;  
let b = 5;
```

```
console.log(a / b);
```

Output:

2

Modulus (%)

Returns the remainder after division.

```
console.log(10 % 3);
```

Calculation:

$10 \div 3 = 3$ remainder 1

Output:

1

Real-Life Use

Check if a number is even or odd.

```
console.log(8 % 2);
```

Output:

0

If remainder is 0, the number is even.

Exponent (**)

Raises a number to a power.

```
console.log(2 ** 3);
```

Calculation:

$2 \times 2 \times 2$

Output:

8

2. Assignment Operators

Used to assign values to variables.

Basic Assignment (=)

```
let age = 20;
```

The value 20 is assigned to age.

Addition Assignment (+=)

```
let score = 10;  
score += 5;  
console.log(score);
```

Equivalent to:

```
score = score + 5;
```

Output:

```
15
```

Subtraction Assignment (-=)

```
let score = 10;  
score -= 3;  
console.log(score);
```

Output:

```
7
```

Multiplication Assignment (*=)

```
let price = 10;  
price *= 2;
```

```
console.log(price);
```

Output:

```
20
```

Division Assignment (/=)

```
let number = 20;
```

```
number /= 4;
```

```
console.log(number);
```

Output:

```
5
```

3. Comparison Operators

Used to compare values.

The result is always:

```
true  
false
```

Equal To (==)

Checks only value.

```
console.log(10 == 10);
```

Output:

```
true
```

```
console.log(10 == "10");
```

Output:

```
true
```

Because == converts types before comparing.

Strict Equal (===)

Checks both value and data type.

```
console.log(10 === 10);
```

Output:

```
true
```

```
console.log(10 === "10");
```

Output:

```
false
```

Because:

```
10      → Number  
"10"    → String
```

Best Practice

Use === whenever possible.

Not Equal (!=)

```
console.log(10 != 5);
```

Output:

```
true
```

Strict Not Equal (!==)

```
console.log(10 !== "10");
```

Output:

```
true
```

Because the data types are different.

Greater Than (>)

```
console.log(10 > 5);
```

Output:

```
true
```

Less Than (<)

```
console.log(10 < 5);
```

Output:

```
false
```

Greater Than or Equal (>=)

```
console.log(10 >= 10);
```

Output:

```
true
```

Less Than or Equal (<=)

```
console.log(5 <= 10);
```

Output:

```
true
```

4. Logical Operators

Used to combine multiple conditions.

Operator Meaning

&& AND

,

! NOT

AND (&&)

Returns `true` only if both conditions are true.

```
console.log(10 > 5 && 20 > 10);
```

Output:

```
true
```

```
console.log(10 > 5 && 20 < 10);
```

Output:

```
false
```

Because one condition is false.

OR (||)

Returns `true` if at least one condition is true.

```
console.log(10 > 5 || 20 < 10);
```

Output:

```
true
```

NOT (!)

Reverses a boolean value.

```
console.log(!true);
```

Output:

```
false
```

```
console.log(!false);
```

Output:

```
true
```

Operator Precedence

JavaScript follows mathematical rules.

Example:

```
console.log(10 + 5 * 2);
```

Output:

20

Calculation:

```
5 * 2 = 10  
10 + 10 = 20
```

Use parentheses when needed.

```
console.log((10 + 5) * 2);
```

Output:

30

Common Beginner Mistakes

Using = Instead of ===

✗

```
if (age = 20)
```

This assigns a value.

✓

```
if (age === 20)
```

This compares values.

Confusing == and ===

```
10 == "10"
```

Output:

```
true  
10 === "10"
```

Output:

```
false
```

Always prefer ===.

Interview Questions and Answers

1. What is an operator?

Answer:

An operator is a symbol that performs an operation on one or more values.

2. What is the difference between == and ===?

Answer:

- == compares values only.
- === compares values and data types.

Example:

```
10 == "10";    // true
10 === "10";   // false
```

3. What does the modulus operator (%) do?

Answer:

It returns the remainder after division.

Example:

```
10 % 3;
```

Output:

```
1
```

4. What is the output?

```
console.log(5 + 2 * 3);
```

Answer:

```
11
```

Because multiplication happens first.

5. What does `&&` mean?

Answer:

AND operator. Returns `true` only when both conditions are true.

6. What does `||` mean?

Answer:

OR operator. Returns `true` if at least one condition is true.

7. What does `!` mean?

Answer:

NOT operator. Reverses a boolean value.

Example:

```
!true
```

Output:

```
false
```

8. What is the output?

```
console.log(10 >= 10);
```

Answer:

```
true
```

9. What is the output?

```
console.log(10 !== "10");
```

Answer:

```
true
```

10. Which comparison operator should you use most often?

Answer:

=== (strict equality operator).

Practice

Predict the output:

```
let a = 10;
let b = 3;

console.log(a + b);
console.log(a % b);
console.log(a > b);
console.log(a === "10");
console.log(a > 5 && b < 5);
```

Try it yourself before moving to Topic 6: **Type Conversion**.

Advanced JavaScript Operators

After learning basic operators, these are the advanced operators commonly used in modern JavaScript.

1. Increment Operator (++)

Increases a value by 1.

```
let count = 5;

count++;

console.log(count);
```

Output:

6

Equivalent to:

```
count = count + 1;
```

Post-Increment (x++)

```
let x = 5;

console.log(x++);
```

```
console.log(x);
```

Output:

```
5  
6
```

Explanation:

- First returns the current value.
 - Then increases it.
-

Pre-Increment (++x)

```
let x = 5;  
console.log(++x);
```

Output:

```
6
```

Explanation:

- First increases.
 - Then returns the new value.
-

2. Decrement Operator (--)

Decreases a value by 1.

```
let count = 5;  
count--;  
console.log(count);
```

Output:

```
4
```

Post-Decrement (x--)

```
let x = 5;  
console.log(x--);
```

```
console.log(x);
```

Output:

```
5  
4
```

Pre-Decrement (--x)

```
let x = 5;  
console.log(--x);
```

Output:

```
4
```

3. Ternary Operator (? :)

Short form of `if...else`.

Normal If-Else

```
let age = 18;  
  
if (age >= 18) {  
  console.log("Adult");  
} else {  
  console.log("Minor");  
}
```

Ternary Version

```
let age = 18;  
  
let result = age >= 18 ? "Adult" : "Minor";  
  
console.log(result);
```

Output:

```
Adult
```

Syntax:

```
condition ? valueIfTrue : valueIfFalse;
```

4. Nullish Coalescing Operator (??)

Returns the right value only when the left value is:

```
null  
undefined
```

Example:

```
let username = null;  
console.log(username ?? "Guest");
```

Output:

```
Guest
```

Important Difference

```
console.log(0 ?? 100);
```

Output:

```
0
```

Because 0 is not null or undefined.

5. Logical OR Assignment (||)

Provides a default value.

```
let username = "";  
username = username || "Guest";  
console.log(username);
```

Output:

```
Guest
```

6. Nullish Assignment (??=)

Assigns a value only if variable is null or undefined.

```
let user = null;  
user ??= "Guest";
```

```
console.log(user);
```

Output:

Guest

7. Logical AND (&&) Short-Circuit

```
let isLoggedIn = true;
```

```
isLoggedIn && console.log("Welcome");
```

Output:

Welcome

If left side is false:

```
false && console.log("Hello");
```

Output:

Nothing

8. Logical OR (||) Short-Circuit

```
console.log(false || "Guest");
```

Output:

Guest

JavaScript returns the first truthy value.

```
console.log("" || 0 || "Mainul");
```

Output:

Mainul

9. Optional Chaining (?.)

Safely accesses object properties.

Without Optional Chaining:

```
let user = null;

console.log(user.name);
```

Output:

Error

With Optional Chaining:

```
let user = null;

console.log(user?.name);
```

Output:

undefined

No error occurs.

10. Spread Operator (...)

Expands arrays or objects.

Array Example

```
let nums1 = [1, 2, 3];
let nums2 = [...nums1, 4, 5];

console.log(nums2);
```

Output:

[1, 2, 3, 4, 5]

Object Example

```
let user = {
  name: "Mainul"
};

let updatedUser = {
  ...user,
  age: 20
};

console.log(updatedUser);
```

Output:

```
{  
  name: "Mainul",  
  age: 20  
}
```

11. Rest Operator (...)

Collects multiple values.

```
function sum(...numbers) {  
  console.log(numbers);  
}
```

```
sum(1, 2, 3, 4);
```

Output:

```
[1, 2, 3, 4]
```

Most Common Interview Questions

What is the difference between `x++` and `++x`?

Answer:

`x++`

Returns current value, then increments.

`++x`

Increments first, then returns value.

What is a ternary operator?

Answer:

A shorthand version of `if...else`.

```
condition ? trueValue : falseValue;
```

What is optional chaining?

Answer:

`?.` allows safe access to object properties without causing errors if the object is `null` or `undefined`.

What is the difference between `||` and `??`?

Answer:

`||` treats all falsy values as missing:

```
0 || 100
```

Output:

```
100
```

`??` only treats `null` and `undefined` as missing:

```
0 ?? 100
```

Output:

```
0
```

What is the difference between Spread and Rest Operators?

Answer:

Spread expands values:

```
[...arr]
```

Rest collects values:

```
function test(...args)
```

At your current JavaScript level, focus mainly on:

1. `++`
2. `--`
3. `? : (Ternary)`
4. `&&`
5. `||`

Learn the others later when you reach ES6+ topic

6. Type Conversion

What is Type Conversion?

Type Conversion means changing a value from one data type to another.

Example:

```
"25" → String  
25   → Number
```

JavaScript often needs to convert data types when performing operations.

Why Do We Need Type Conversion?

Imagine a user enters their age in a form:

```
let age = "20";
```

The value is a **string**, not a number.

If you want to do math:

```
console.log(age + 5);
```

Output:

```
205
```

JavaScript joins the values instead of adding them.

To get the correct result:

```
let age = Number("20");  
  
console.log(age + 5);
```

Output:

```
25
```

Types of Type Conversion

There are two types:

1. Implicit Type Conversion (Automatic)
2. Explicit Type Conversion (Manual)

1. Implicit Type Conversion

JavaScript automatically converts data types when needed.

This is also called **Type Coercion**.

String + Number

```
console.log("10" + 5);
```

Output:

105

Why?

JavaScript converts:

5 → "5"

Then:

"10" + "5"

Result:

105

Number - String

```
console.log(10 - "5");
```

Output:

5

JavaScript converts:

"5" → 5

Then:

10 - 5

Result:

Multiplication

```
console.log("10" * 2);
```

Output:

20

JavaScript converts:

"10" → 10

Division

```
console.log("20" / 2);
```

Output:

10

Strange Examples

Boolean to Number

```
console.log(true + 1);
```

Output:

2

Because:

```
true = 1
```

```
console.log(false + 1);
```

Output:

1

Because:

```
false = 0
```

2. Explicit Type Conversion

You manually convert values.

Convert to Number

Using `Number()`.

```
let age = "20";  
  
let result = Number(age);  
  
console.log(result);
```

Output:

20

Check Type

```
let age = "20";  
  
console.log(typeof age);
```

Output:

string

After conversion:

```
let age = Number("20");  
  
console.log(typeof age);
```

Output:

number

Invalid Conversion

```
Number("Hello");
```

Output:

NaN

What is NaN?

NaN means:

Not a Number

Example:

```
console.log(Number("JavaScript"));
```

Output:

NaN

Convert to String

Using `String()`.

```
let age = 20;
let result = String(age);
console.log(result);
```

Output:

"20"

Type:

```
console.log(typeof result);
```

Output:

string

Convert to Boolean

Using `Boolean()`.

Truthy Values

These become `true`.

```
Boolean(1)
```

```
Boolean(100)
Boolean("Hello")
Boolean("JavaScript")
```

Output:

```
true
```

Falsy Values

These become `false`.

```
Boolean(0)
Boolean("")
Boolean(null)
Boolean(undefined)
Boolean(NaN)
```

Output:

```
false
```

Important Concept: Truthy and Falsy

JavaScript treats some values as true and some as false.

Falsy Values (Interview Favorite)

There are only a few falsy values:

```
false
0
""
null
undefined
NaN
```

Everything else is generally truthy.

Example

```
if ("Mainul") {
  console.log("Runs");
}
```

Output:

Runs

Because "Mainul" is truthy.

Example

```
if ("" ) {  
  console.log("Runs");  
}
```

Output:

Nothing

Because an empty string is falsy.

parseInt()

Converts a value to an integer.

```
console.log(parseInt("25"));
```

Output:

25

With Decimal

```
console.log(parseInt("25.99"));
```

Output:

25

Decimal part is removed.

parseFloat()

Converts a value to a decimal number.

```
console.log(parseFloat("25.99"));
```

Output:

```
25.99
```

Number() vs parseInt()

Number()

```
Number("25.99");
```

Output:

```
25.99
```

parseInt()

```
parseInt("25.99");
```

Output:

```
25
```

Common Beginner Mistakes

Mistake 1

```
let age = "20";  
console.log(age + 5);
```

Output:

```
205
```

Not:

```
25
```

Correct

```
let age = Number(age);  
console.log(age + 5);
```

Output:

25

Mistake 2

```
Boolean("false");
```

Output:

true

Why?

Because "false" is a non-empty string.

Any non-empty string is truthy.

Interview Questions and Answers

1. What is type conversion?

Answer:

Type conversion is the process of converting a value from one data type to another.

2. What are the two types of type conversion?

Answer:

1. Implicit Type Conversion
 2. Explicit Type Conversion
-

3. What is type coercion?

Answer:

Automatic type conversion performed by JavaScript.

Example:

"10" + 5

Output:

105

4. What does `Number ()` do?

Answer:

Converts a value into a number.

Example:

```
Number ("20") ;
```

Output:

20

5. What is NaN?

Answer:

NaN stands for:

Not a Number

It appears when JavaScript cannot convert a value into a valid number.

6. What are falsy values in JavaScript?

Answer:

```
false  
0  
""  
null  
undefined  
NaN
```

7. What is the output?

```
console.log ("10" + 5);
```

Answer:

105

8. What is the output?

```
console.log("10" - 5);
```

Answer:

5

9. Difference between `parseInt()` and `parseFloat()`?

Answer:

```
parseInt("25.99")
```

Output:

```
25  
parseFloat("25.99")
```

Output:

25.99

10. What is the output?

```
console.log(Boolean(""));
```

Answer:

false

Practice

Predict the output:

```
console.log("20" + 10);  
console.log("20" - 10);  
console.log(Number("50"));  
console.log(Boolean(""));  
console.log(Boolean("Hello"));  
console.log(parseInt("99.99"));  
console.log(parseFloat("99.99"));
```

7. Template Literals

What are Template Literals?

Template Literals are a modern way to create strings in JavaScript.

They allow you to:

- Insert variables into strings easily
- Create multi-line strings
- Write cleaner and more readable code

Template literals were introduced in **ES6 (ECMAScript 2015)**.

Backticks (`)

Template literals use **backticks** instead of single or double quotes.

```
let name = `Mainul`;
```

Backtick key location:

```
` ← Usually below ESC and above TAB
```

Why Were Template Literals Introduced?

Before template literals, combining strings and variables was messy.

Old Way (String Concatenation)

```
let name = "Mainul";  
let age = 20;  
  
let message = "My name is " + name + " and I am " + age + " years old.";  
  
console.log(message);
```

Output:

```
My name is Mainul and I am 20 years old.
```

This works, but becomes difficult to read as strings get longer.

String Interpolation

Interpolation means inserting variables or expressions inside a string.

Template literals use:

```
${}
```

Syntax:

```
`Text ${variable}`
```

Example

```
let name = "Mainul";  
console.log(`My name is ${name}`);
```

Output:

```
My name is Mainul
```

Multiple Variables

```
let name = "Mainul";  
let age = 20;  
console.log(`My name is ${name} and I am ${age} years old.`);
```

Output:

```
My name is Mainul and I am 20 years old.
```

Expressions Inside `${}`

You can put JavaScript expressions inside `${}`.

Addition

```
let a = 10;  
let b = 5;  
console.log(`Sum = ${a + b}`);
```

Output:

```
Sum = 15
```

Multiplication

```
console.log(`Result = ${10 * 5}`);
```

Output:

```
Result = 50
```

Comparison

```
let age = 20;

console.log(`Adult: ${age >= 18}`);
```

Output:

```
Adult: true
```

Calling Functions Inside `${}`

```
function greet() {
  return "Hello";
}

console.log(`${greet()} Mainul`);
```

Output:

```
Hello Mainul
```

Multi-Line Strings

Before template literals:

```
let text = "Line 1\nLine 2\nLine 3";

console.log(text);
```

Output:

```
Line 1
Line 2
Line 3
```

Notice the `\n`.

With template literals:

```
let text = `
Line 1
```

```
Line 2
Line 3
`;

console.log(text);
```

Output:

```
Line 1
Line 2
Line 3
```

No `\n` needed.

Real-World Example

Suppose you're building a student profile.

```
let name = "Mainul";
let age = 20;
let country = "Bangladesh";

let profile = `
Name: ${name}
Age: ${age}
Country: ${country}
`;

console.log(profile);
```

Output:

```
Name: Mainul
Age: 20
Country: Bangladesh
```

Template Literals vs String Concatenation

Concatenation

```
let name = "Mainul";

console.log("Hello " + name);
```

Template Literal

```
let name = "Mainul";

console.log(`Hello ${name}`);
```

Both work, but template literals are cleaner.

Common Beginner Mistakes

Using Quotes Instead of Backticks



```
let name = "Mainul";  
console.log("Hello ${name}");
```

Output:

```
Hello ${name}
```

Because JavaScript treats it as normal text.



```
let name = "Mainul";  
console.log(`Hello ${name}`);
```

Output:

```
Hello Mainul
```

Forgetting \${}



```
let name = "Mainul";  
console.log(`Hello name`);
```

Output:

```
Hello name
```



```
console.log(`Hello ${name}`);
```

Output:

Hello Mainul

Interview Questions and Answers

1. What is a Template Literal?

Answer:

A Template Literal is a string enclosed by backticks (` ) that allows variable and expression interpolation using `${}`.

2. Which symbol is used for Template Literals?

Answer:

Backticks:

`

Not:

"

'

3. What is String Interpolation?

Answer:

String interpolation is the process of inserting variables or expressions into a string using `${}`.

Example:

```
let name = "Mainul";  
console.log(`Hello ${name}`);
```

4. What is the output?

```
let name = "Mainul";  
console.log(`Hello ${name}`);
```

Answer:

5. Can we use expressions inside `${}`?

Answer:

Yes.

Example:

```
console.log(`${10 + 5}`);
```

Output:

```
15
```

6. What is the output?

```
let a = 10;  
let b = 5;
```

```
console.log(`Result = ${a * b}`);
```

Answer:

```
Result = 50
```

7. What is the advantage of Template Literals?

Answer:

- Easier string interpolation
 - Cleaner code
 - Supports multi-line strings
 - More readable than string concatenation
-

8. What is the output?

```
console.log(`Age: ${20}`);
```

Answer:

```
Age: 20
```

9. Can functions be used inside `${}`?

Answer:

Yes.

Example:

```
function greet() {  
  return "Hello";  
}  
  
console.log(`${greet()} Mainul`);
```

Output:

Hello Mainul

10. What is the difference between concatenation and template literals?

Answer:

Concatenation:

```
"Hello " + name
```

Template Literal:

```
`Hello ${name}`
```

Template literals are cleaner and easier to read.

Practice

Predict the output:

```
let name = "Mainul";  
let age = 20;  
  
console.log(`Name: ${name}`);  
console.log(`Age next year: ${age + 1}`);  
console.log(`Adult: ${age >= 18}`);  
console.log(`Sum: ${10 + 20}`);
```

8. User Input

What is User Input?

User Input means getting information from the user while the program is running.

Examples:

- User enters their name
- User enters age
- User enters email
- User enters password
- User enters product quantity

Without user input, programs cannot interact with users.

Why is User Input Important?

Imagine a website asks:

What is your name?

The user types:

Mainul

The program receives that value and can use it.

This is called **User Input**.

Taking Input in JavaScript

For beginners, the easiest way is using:

```
prompt()
```

Syntax:

```
prompt("Message")
```

Example 1

```
let name = prompt("Enter your name");  
console.log(name);
```

If user enters:

Mainul

Output:

Mainul

How `prompt()` Works

```
let name = prompt("Enter your name");
```

Step 1:

Browser shows:

Enter your name

Step 2:

User types:

Mainul

Step 3:

JavaScript stores the value in:

name

Example 2

```
let city = prompt("Enter your city");  
console.log(`City: ${city}`);
```

Input:

Dhaka

Output:

City: Dhaka

Important: `prompt()` Always Returns a String

This is a very common interview question.

Example:

```
let age = prompt("Enter age");
```

```
console.log(typeof age);
```

If user enters:

20

Output:

string

Not:

number

Because `prompt()` always returns a string.

Problem Example

```
let age = prompt("Enter age");
```

```
console.log(age + 5);
```

Input:

20

Output:

205

Why?

Because:

"20" + 5

becomes:

"20" + "5"

Result:

205

Convert Input to Number

Use:

```
Number()
```

Example:

```
let age = Number(prompt("Enter age"));  
console.log(age + 5);
```

Input:

20

Output:

25

Now it works correctly.

Using parseInt()

```
let age = parseInt(prompt("Enter age"));  
console.log(age);
```

Input:

20

Output:

20

Type:

number

Using parseFloat()

```
let price = parseFloat(prompt("Enter price"));  
console.log(price);
```

Input:

99.99

Output:

99.99

Example: Sum of Two Numbers

```
let num1 = Number(prompt("Enter first number"));
let num2 = Number(prompt("Enter second number"));

let sum = num1 + num2;

console.log(sum);
```

Input:

10
5

Output:

15

Example: Calculate Age Next Year

```
let age = Number(prompt("Enter your age"));

console.log(`Next year you will be ${age + 1}`);
```

Input:

20

Output:

Next year you will be 21

Cancel Button

When user presses **Cancel**:

```
let name = prompt("Enter your name");
```

Result:

null

Example:

```
console.log(prompt("Enter your name"));
```

User presses Cancel.

Output:

```
null
```

Checking Input Type

```
let value = prompt("Enter something");
```

```
console.log(typeof value);
```

Whatever the user types:

```
string
```

unless they press Cancel, which returns:

```
object
```

because:

```
typeof null
```

returns:

```
object
```

(known JavaScript historical bug)

Common Beginner Mistakes

Mistake 1

```
let age = prompt("Enter age");
```

```
console.log(age + 10);
```

Input:

```
20
```

Output:

```
2010
```

Not:

30

Because age is a string.

Correct

```
let age = Number(prompt("Enter age"));  
console.log(age + 10);
```

Output:

30

Mistake 2

```
let age = Number(prompt("Enter age"));  
console.log(typeof age);
```

Input:

abc

Output:

number

Value:

NaN

Remember:

NaN

still has type:

number

Interview Questions and Answers

1. What is user input?

Answer:

User input is data provided by the user while a program is running.

2. Which function is commonly used to take input in the browser?

Answer:

`prompt()`

3. What does `prompt()` return?

Answer:

A string.

Example:

```
let age = prompt("Enter age");
```

Even if user enters:

20

the value is:

"20"

4. What is the output?

```
let age = prompt("Enter age");  
console.log(typeof age);
```

Answer:

string

5. How do you convert prompt input into a number?

Answer:

```
Number(prompt("Enter age"))
```

6. What is the difference between `Number()` and `parseInt()`?

Answer:

```
Number("20.5")
```

Output:

```
20.5  
parseInt("20.5")
```

Output:

```
20
```

7. What happens if the user clicks Cancel?**Answer:**

prompt() returns:

```
null
```

8. What is the output?

```
let age = Number(prompt("Enter age"));  
console.log(age + 5);
```

Input:

```
20
```

Output:

```
25
```

9. What is NaN?**Answer:**

NaN means:

```
Not a Number
```

It appears when a value cannot be converted into a valid number.

10. What is the output?

```
console.log(Number("abc"));
```

Answer:

NaN

Practice

Predict the output:

```
let num1 = Number(prompt("Enter first number"));
let num2 = Number(prompt("Enter second number"));
```

```
console.log(num1 + num2);
console.log(num1 * num2);
```

If the user enters:

10
5

What will be printed?